

Preface — the operator’s account

Erik Garrison, September 2026. Dictated in a single agent thread, without the orchestration framework running — fitting, since the same cool-down applies to how this record ends. I have edited only for flow; the voice is mine.

I built wg because I wanted the work to hold itself. The concept is a work graph — a graph of work. The implementation is worksgood, which is what you enter on the command line.¹ I was trusting the model to be able to do everything, and the policy layer was very, very thin — and for the first six months of its life, the system was working fantastically. I was able to do incredible things, performance of very complex tasks, precisely *because* nothing stood between me and the model’s capability.

Then I started using weaker models, and they began to violate the constraints of the system, bringing down its operation. A lot of the policy stuff and failures came out of trying to deal with low-reliability models — operational reliability and also intellectual reliability — on OpenRouter. The reaction was: these are failing, let me get a bug report from the system, bring it back into the wg directory, and have the agents and agency work on it and try to fix it. Which they did. Each time.

There’s this whole problem of agency itself. Agency was a concept — still a concept I’d like to explore — but it’s become much more diffused, and its utility much less clear, because it really looks like you don’t need it in many contexts. There’s so much in-context learning happening that you get it for free as long as the agents are exposed to their operational context — code generation, document generation, communication, research, whatever they’re doing. Telling agents about a skill reminds them they have it, but they don’t need that if they’re in the right context. They fall into the right set of skills and behaviors on their own.

So what happened to me, in the middle of all this: I dug in really deep. Things weren’t working, and I was like, okay, I’m going to try to fix it. So I kept adding more and more stuff. The complicated evaluation system became harder and harder for agents to pass. I kept thinking I’d fixed it. Throughout the summer I kept using it, kind of in control, and in small systems it would work okay. But then the final straw: observing that things would *work* but that they were failing over and over, requiring huge numbers of interventions from myself.

You end up in a place where the graph does all the things it should do. It holds information about the work. The machinery is all working. But every task fails. Every single one. You poke it and it’s slightly fixed. You poke it again and it’s slightly fixed. In the end you manually say, oh well, it passed, I’ll just mark it as passed. And you get to: I can’t understand what is going on. Why doesn’t this work? That’s when I ran down the phenomenon in the documentation — the bureaucratization cascade — and recognized it.

I can name the mechanism now, the way it felt from the inside. Things actually worked okay until eventually there were so many constraints that, even if the probability of failure in any one of them is low — if you have a 1% failure on a constraint and you have 100 constraints, you begin to have an expectation that you’re always going to fail.

The resolution is as described in this book. We haven’t removed a lot of these things. We’ve stripped their power out of the system.

¹ Annoyingly, I discovered later that wg is how you invoke WireGuard — which, through Tailscale, is how I actually connect to the host I’m working on. The collision is real and I choose to find it funny.

The Book of WG

The life, death, and recovery of an AI agent organization — told from its own record

Assembled September 2026 from the git history, coordination ledgers, configuration backups, quarantined state, and the operator’s prompting record. Every dated claim traces to the repository record. Uncertainties are flagged, not resolved by inference. The companion paper is Bureaucratization Cascades in Hybrid Human–AI Organizations (bureaucratization.github.io).

The arc in one paragraph

On January 18, 2026, a developer bootstrapped a task-graph library and, by January 24, its own development roadmap. Over nine months the agents that built it accumulated 312 constraint topics and removed one; their durable archive died in a June substrate crisis the daemon logged 76,371 times into a log nobody read; the organization re-founded itself on July 4 (the “overengineered-cutover”) and ignited a ten-fold constraint spike; recovery began with the August 5 atomic quarantine of the damaged graph and ran as a five-week subtraction arc—the August 7 teardown, the August 9 rescue manifesto, the August 10 demotion of evaluators to witnesses—until the September 13 enforcement-autonomy disable and the September 14 constitution completed it. The machinery was then retired, its model registry externalized to the host harness, and forward work resumed without the burden signature.

Master timeline

- **Jan 18–19** — Born twice: the library, then self-hosting (“workgraph now tracks its own development roadmap”)
 - **Feb 1–27** — The three seeds planted: verification workflows (Feb 1), the Agency system (Feb 6, the same day `.workgraph` enters `.gitignore`), autopoietic task generation (Feb 27)
 - **Mar 3–5** — FLIP is born (scoring + Opus-triggered verification)
 - **Apr 5–11** — Verify gates born in the triage week; credit-exhaustion abandonments; the ~95-constraint burst; 1,558 tests green at peak
 - **May–Jun** — The quiet (operator running two grant campaigns elsewhere, 93–95% completion) and the substrate crisis: corruption hardening (Jun 2), the `workgraphworksgood` rename (Jun 16), the graph dies (Jun 18, 11:48 — 76,371 unobserved ticks), divergent mains reconciled (Jun 26); executor work consumes 47% of the window’s commits
 - **Jul 4–11** — The churn week: the graph recreated, agency automation switched on, eight route states in eight days, the constraint spike (~28 defensible topics)
 - **Aug 1–5** — Surprise mass-archive held (Aug 1), auto-archive made opt-in (Aug 2), atomic-save designed (Aug 3), disk exhaustion (Aug 5) triggers the atomic quarantine tagged `recovery/overengineered-cutover-20260805`
 - **Aug 7–10** — The teardown day (47 subtractive commits), the rescue manifesto (357 lines), evaluators demoted to witnesses; fresh clone from GitHub (Aug 7); current ledger begins (Aug 8)
 - **Aug 20–Sep 1** — Convalescence: one commit in twelve days
 - **Sep 2–6** — The rebuild sprint: project-local Pi config, the Sol/Luna re-route
 - **Sep 13–16** — Enforcement autonomy disabled; completions 935/week; the constitution (Sep 14); registry retired and forward work resumes (Sep 16)
 - **Standing** — Automation stays off, gates concierge-attended, burden signature absent
-

Cast

- **The operator** — Erik Garrison. Router of pain, asker of “wdyt?”, author of the manifesto and the constitution-era design docs; the bankruptcy mechanism
- **The machinery** — Agency (b. Feb 6, d. demobilized Sep 13; survived by atomic saves, a concierge, and a written constitution), FLIP (b. Mar 3, retired Sept), the verify gates (b. Apr 5–11)

- **The executors** — a zoo of nine species tried in six months: amplifier, aider, claude, cline, codex, crush, goose, opencode, qwen — then pi, the answer that arrived as a question (“thinking we should test out ... ‘pi’. wdyt?”)
- **The agents** — renumbered from agent-1 on July 3; 3,700 self-given names in the wiki era, task-suffix identities in the commit era; the narrator-mode strain that produces plans instead of work (observed twice in this book’s own authoring fleet)

Chapter 1 — Bootstrap (January 18 – February 28, 2026)

Every dated claim in this chapter comes from the wg commit log itself (git log main, Jan 18 – Feb 28, 2026, 258 commits). Quotes are verbatim from commit subjects or bodies, trimmed to 200 characters.

Day zero, twice

The repository’s first three commits land on **January 18, 2026**, and they are a project in miniature:

1. 0d06613d — “Initial workgraph library with core types, parser, queries, and checks”
2. 70064848 — “Add CLI with init, add, done, ready, blocked, check, list, graph, cost”
3. 8b4b523c — “Add research documentation and project notes”

A library, a CLI, and notes — by a human author (Erik Garrison; all 192 commits in this window are his). The next day comes the commit that makes the project unlike any other in the record: **56e1ae70** — **“Bootstrap workgraph with its own development tasks + Claude skill.”** The commit body states it without decoration: *“Self-hosting: workgraph now tracks its own development roadmap.”* The same day adds **7a86766e** — “Add CLAUDE.md with workgraph agent protocol” — the first constitution: a written protocol for how agents would enter and work in the repository.

So day two is the day the tool began tracking its own construction. Every subsequent commit in the record is the system developing itself, under one human’s direction.

The first week: a CLI grows a vision (Jan 19–24)

January 19 is a burst of fourteen commits — analysis commands in three phases, timestamp scheduling, a **plan** command (“wg plan –budget X –hours Y”), a **coordinate** command for parallel dispatch, claim/unclaim commands for agent coordination, resource and actor commands, and the first research documents (“Add human interface research document,” “Add human summoning and TUI design research”). The pattern established on day two — build the tool, then write the research note — is already visible.

January 20 delivers two structural decisions in a single day: **324fc5b0** — “Rename binary from ‘workgraph’ to ‘wg’” — the binary’s name shortens three weeks before the package itself would (June’s rename is prefigured here); and **13685266** — “Design workgraph path to autonomous agents” — the roadmap document that the following days execute.

January 21–24 is a deliberate construction sequence, each commit a named layer:

- Jan 21: “Implement Layer 0: foundation tasks”; “Implement Layer 1: task metadata and failure handling”
- Jan 23: “Implement agent-identity: capabilities, trust levels, matching”; “Implement context-inheritance: artifacts and context flow”; “Implement agent-architecture: autonomous work loop support”
- Jan 24: “Implement trajectory-claiming: context-efficient task chains”; “Implement execution-model: coordination-only with exec helper”; and finally — with a celebratory emoji, the only one in the early record — **“Implement autonomous-runtime: the goal task!”**

In seven days the project goes from three commits to an autonomous runtime. The “goal task” — a task whose purpose is to spawn and coordinate further tasks — is the seed of everything that follows: the June swarms, the July burden, the September constitution all descend from this commit’s idea.

Late January: the agent loop closes (Jan 26 – Feb 3)

After the runtime lands, the record pauses (no commits Jan 25–30) and resumes with infrastructure for the agents who will use it: spawn prompt fixes, model selection, coordinator detection of finished agents (Jan 31); then the first **verification workflow** (Feb 1: “Add verification workflow for tasks,” “Add wrapper script to auto-complete tasks when agents exit”). The first governance — a verification step — arrives three weeks in, gently.

February 2 is a consolidation day: the coordinator merges into the service daemon with event-driven task pickup; the interactive TUI dashboard lands (`wg tui`); service daemon logging and hot-reload arrive. And one quiet commit with long consequences: **“Stop tracking workgraph runtime state files”** — the runtime state leaves git’s view on February 2, four months before the state’s own survival becomes the story’s crisis.

February 3 adds the matrix-lite Matrix client (the first communication substrate), a DAG layout engine, `wg quickstart/wg status`, and the first skill-install command. February 6 closes the month’s arc with two commits that define the next phase: “Make wg init add .workgraph to .gitignore” (the state directory officially becomes untracked — a decision whose costs arrive in §4.5 of the paper) and **“Implement Agency system: evolutionary agent identities for workgraph”** — the first appearance of the word *agency* in the record.

February: audits, ADRs, and the org-eval experiment (Feb 6–28)

Mid-February is dominated by audit cycles: fourteen consecutive “Audit cycle cleanup/fixes” commits (Feb 14–15), addressing clippy warnings, dead code, error handling, Unicode truncation, shell escaping, atomic saves — “add validation, atomic saves, better diagnostics” appears on **Feb 15**, nearly five months before atomic saves would be rebuilt as a response to the June graph loss. The hygiene instinct was present from the start; it simply was not a mechanism.

The late-February record is an explosion of ambition: 114 commits in the final two weeks of the month. February 24–26 brings ASCII DAG visualization with box-drawing characters, a TUI search UX, token usage display with “in/out/val slash notation,” a four-layer LLM test suite (“snapshots, contracts, mock executor, Haiku smoke”), typst documentation conversion, and `57489e2a` — “wg setup auto-installs skill/bundle for chosen executor” — the first executor-choice plumbing.

Then the agency experiment goes vertical. February 26–27 delivers, in order: “agency system overhaul, TUI viz viewer, service hardening”; “propagate evaluations to role components”; “unified evaluation with org dimensions”; a CLAUDE.md directive — “Task tool prohibition, orchestrating agent role”; and on February 27 alone: “compose special agents at agency init,” “proper scoring rules and rubric spectrum for evaluations,” “autopoietic task generation with guardrails,” “agency audit, gc/link/tradeoff/create commands, exec_mode,” and “coordinator org-eval, assigner evaluation, agency create command.”

The vocabulary shift is the story of the fortnight: by February 27 the record speaks of *evolutionary agent identities*, *org-evaluations*, *assigner scoring*, *rubric spectra*, *tradeoffs*, and *autopoietic task generation*. The tool is no longer a task graph; it is becoming an **evolutionary agent organization** — with scoring rules, special agents composed at init, and evaluation recorded “against special agent entity.” One commit on Feb 27 — “refactor: remove org-eval infrastructure” — suggests the first experiment was already being torn out the day after its biggest expansion: the accrete-and-remove cycle, present in miniature from the very first month, at a scale of days rather than months.

The window closes (Feb 28) with improvement-loop commits — “improvement-loop-1: extract evolve module... clean dead code” through “improvement-loop-3: split spawn/main modules” — the first recorded *improvement loops*, an explicit self-improvement pattern that foreshadows the June autopoietic loop and its failure report.

The first 30 days

- **Jan 18:** library, CLI, notes. **Jan 19:** self-hosting begins; the agent protocol written.
- **Jan 21–24:** layered construction to an autonomous runtime with a “goal task.”
- **Jan 31–Feb 6:** the agent loop closes (spawn, verify, daemon, TUI), the state leaves git, and the **Agency system** appears.

- **Feb 14–15:** fourteen audit-cycle hygiene commits — including the first “atomic saves.”
- **Feb 24–28:** visualization, four-layer LLM tests, executor-choice setup, and the org-eval/agency explosion — with its first same-week teardown.

The cast is one human (all 192 commits authored by Erik Garrison) and, from late February, an expanding population of special agents with component-based identities and scoring rubrics. The governance count at window’s end is small — the accretion has not begun in earnest — but three of its future ingredients are already committed to the record: verification workflows (Feb 1), the Agency system (Feb 6), and autopoietic task generation (Feb 27). The needle exists on day two. It will take five months to close.

Timeline — Chapter 1

- **Jan 18:** workgraph born — library, CLI, notes (first 3 commits)
 - **Jan 19:** self-hosting: the tool tracks its own roadmap; agent protocol written
 - **Jan 24:** “Implement autonomous-runtime: the goal task!” — the autonomous runtime lands
 - **Feb 6:** Agency system (evolutionary agent identities); `.workgraph` added to `.gitignore`
 - **Feb 26–28:** org-eval/agency explosion, autopoietic task generation; first same-week teardown; 258 commits in the window
-

Chapter 2 — Peak and Triage (March–April 2026)

March: polish, deliberation, and the first verify-fixes

March opened quietly relative to what followed: 765 commits in the month, opening with maintenance and self-observation. On March 9–10 the record shows an organization studying itself — eval agents taught to record token usage, aggregated agency token usage (the marker) surfaced on parent tasks, a coordination-model design document, and a “design deliberation consensus document” produced through a discussion-host facilitation task. The org was already running multi-agent deliberations about its own design.

The first hardening fingerprints also appear in March, in miniature: `.verify-implement-tui-inspector-2` produced three consecutive fix commits (pane-size enforcement, minimum-pane enforcement, FD-exhaustion resilience in watcher tests), and `.verify-fix-cycle-aware` fixed back-edge detection with sorted iteration. These are the first visible instances of the verifyfixverify loop that would later compound — at this scale, single-task and single-cycle, they read as diligence rather than pathology.

The dominant March work was interface: TUI, chat, Telegram. A March 10 perf commit cut wg-viz idle CPU from ~50% to under 5% — the kind of fix that only happens when someone is actually staring at the dashboard, which, given the human prompting record for this period, they were.

April: the peak, and what the peak was made of

April produced 1,082 commits — the organization’s all-time maximum. The busiest days were April 12 (96 commits) and April 11 (81), which are also the triage days. Agent numbering passed 1,318 by month’s end; 224 distinct agent IDs appear in the two-month window, though most committed once or twice — the distribution is long-tailed, with agent-1 (5 commits) the most frequent committer in main’s record.

The peak was not one project. A theme decomposition of the window’s commit subjects:

- ~1,337 unclassified feature/fix work (the bulk: TUI, chat, html surfaces, publisher)
- ~341 TUI/chat/Telegram interface commits
- ~100 terminal-bench (TB) evaluation and evolution commits
- ~30 audit commits, ~25 FLIP commits

The TB campaign is the most distinctive sub-project. Beginning April 3–4 (“TB trials fanout pipeline — create wg tasks from TB definitions”), the organization ran a full $4 \times 7 \times 3$ evaluation sweep — four conditions, seven tasks, three replicas — across models including GPT-OSS-120B and Claude Haiku, then built an **evolution**

pipeline: “evolution run-01 report — 18 ops applied from full-sweep-01 data,” “rerun worst performers from full-sweep-01 with evolved agents,” and a “FLIP accuracy as false-PASS detector” investigation. On April 5 the org audited its own availability and found itself “available but under-prompted.” This is an agent organization running controlled experiments on its own agents and applying the results — the evolver subsystem in full flight.

April 12 also planted a seed that would grow for months: `impl-prevent-self-failure - add anti-self-failure guardrails to agent prompts`. This is the earliest explicit self-protection measure in the record, and it lives in the prompts — the layer the operator could not see.

The triage: April 11

The `FINAL_TRIAGE_REPORT` is dated April 11 (agent-15023, “document-final-triage”). Its context: 96 and 81 commits on the surrounding days, the TB sweep running, and the system under visible strain. The triage categorized 8 failed tasks:

Retried (3): `fix-ci-and` (agent failure during legitimate CI work), `fix-add-command-2` (timeout mid-way through function-signature fixes, 6/10 done), `make-eval-failure` (work completed but the verify command was malformed).

Abandoned (5): `.flip-fix-ci-and` and `.evaluate-fix-ci-and` (402 credit exhaustion, minimax/minimax-m2.7), `.verify-implement-monotonic-coordinator` (agent killed, exit 143 — likely credit exhaustion), `.verify-investigate-coordinator-resurrection` (circuit breaker on compilation errors), `verify-autopoietic-independence` (circuit breaker).

At the same moment the record shows 31 orphaned evaluation/FLIP tasks cleaned up and 1,558 tests passing. The system was simultaneously at maximum productivity and showing every failure signature it would later become known for: credit exhaustion, circuit breakers, killed agents, verify-command malformations, orphaned evaluations.

The constraint births accelerated in step: 149 commits in the window match governance vocabulary (contract/gate/verify/proof/admission), consistent with the census’s April figure (~95 constraint topics born, the year’s highest month).

Reading the peak

Three things stand out from the traces, and one honest uncertainty.

First, the peak was healthy. The April 11 triage found all compilation errors resolved, the dependency graph intact, and the full test suite passing. Whatever strain existed, the org shipped through it — including the TB infrastructure and the evolution pipeline.

Second, the failure signature was already complete in miniature. Every failure mode that dominates the later record — credit exhaustion, timeouts, malformed verify commands, circuit breakers, orphaned evaluations — appears in the April 11 triage. What changed later was not the kinds of failure but the response: in April, failures were triaged and work continued; by summer, the same failures would each leave behind a permanent rule.

Third, the self-protective turn began here, quietly. The April 12 anti-self-failure guardrails went into agent prompts — the layer with no operator visibility — and the verify-timeout machinery that would later generate its own escalation ladder (`fix-verify-timeout-default`, `WG_VERIFY_TIMEOUT`, the `timeout-eval` tests) started with a single 120s300s bump on April 5.

The uncertainty: commit subjects carry task names, not full attribution. The 224-agent count and per-agent frequency are derived from subject suffixes and merge records; the true number of distinct agent instances may differ. And the “95 constraint births” figure comes from the gate census’s topic clustering, which is an upper bound (§5 of the paper) — the vocabulary-matched count of 149 commits is the looser measure.

Timeline

- **2026-03-09/10** — Self-observation era: token-usage tracking, coordination-model design doc, deliberation consensus document
 - **2026-03 (month)** — 765 commits; interface polish (TUI/chat/Telegram); first verifyfix loops appear at single-task scale
 - **2026-04-03/04** — Terminal-bench trials fanout begins; evolution pipeline created
 - **2026-04-05** — FLIP false-PASS investigation; verify timeout raised 120s300s; “wg available but under-prompted”
 - **2026-04-11/12** — The triage (8 failed tasks; 402 credit exhaustion; circuit breakers) on the two busiest days of the project’s life; anti-self-failure guardrails added to agent prompts
-

Chapter 3 — The Quiet and the Substrate Crisis

May 1 – June 30, 2026 · 324 commits on main in the window (approximately 130 in May and 190 in June by author date — the paper’s monthly table uses committer date and differs by a few commits — against April’s 1,089)

3.1 The quiet

The primary repository went quiet in May. Not because the organization failed — because its operator was elsewhere. Two grant campaigns ran through *other* wg deployments on the same machine in exactly this window: *grant campaign B* (352 tasks, May 12–29, 93% completion, including a 123-of-137 day) and *grant campaign A* (848 tasks, May 19–June 5, ~95% completion). The tool was working; the operator was working; the primary repo simply received less attention. Later reconstruction would show the same pattern in the human prompt record: near-silence for the primary repo in May (16 prompts), against ~1,000 turns across the whole study.

This matters for everything that follows. May–June is *not* a collapse; it is the calm before the ignition — and, as it turned out, the period in which the organization’s own record of itself died.

There is one more quiet-month record, and it closes the loop. The wg repository itself kept developing during this window — on the operator’s laptop, through a separate wg deployment whose graph survived intact: **1,466 tasks created April–May at *declining* multi-dispatch (6.4%3.1%)**, 90% completed with a real failure tail, and flat average dispatch (~0.85). The same development work, the same tool, the same operator — moved to another machine — ran healthy. The primary’s problem was never the tool, the executor era, or the period. It was the primary’s accumulated rulebook.

3.2 The executor zoo

The main technical story of May–June is not governance. It is **executor and model churn** — the systematic attempt to find a better way to run the agents. The commit record names at least four executor lineages in play, plus a menagerie of configuration templates:

- **nex** — a custom CLI/executor that dominated this window (~280 commits touching it), with its own streaming client, cursor handling, and OpenRouter endpoints: `nex-optional-openrouter-endpoint`, `nex-openrouter-default-ui`, `bench-nex-minimax-terminal`, `feat(tui): native executor now runs interactive wg nex REPL in PTY`.
- **opencode** — researched and exposed June 11–14 (`research-opencode-executors`, `expose-opencode-openrouter`, `fix-opencode-per-task-routing`).
- **codex** — the incumbent; still the daemon’s declared executor in the death window (§3.5).
- **pi** — the newcomer whose migration sprint fills June 17–23 (`design-systematic-pi`, `implement-pi-plugin`, `fix-wg-pi`, `fix-pi-model`, `fix-pi-handler`, `design-handler-first`, and on June 22 the verdict commit `feat: pi-plugin-research - verdict + wg-pi-plugin design`).

The template directory of the era (preserved in the August 5 quarantine) lists nine executor recipes: `aider`, `amplifier`, `claude`, `cline`, `codex`, `crush`, `goose`, `opencode`, `qwen`. Model routing ran over OpenRouter with per-task endpoint overrides; a minimax model was benchmarked (`bench-nex-minimax-terminal`); a “model-scout” was built to tune the choice (`bug-model-scout-tuning-not-installed`). Across the organization’s whole life, provider/executor migration would account for 430 commits — **13% of everything** — and most of that tax was paid in this window.

3.3 Corruption was already a known hazard

Well before anything broke publicly, the record shows the runtime state as a fragile surface:

- **May 2** — feat: `fix-nex-cursor-corruption` (agent-1799).
- **Late May** — design documents themselves had to be restored from prior revisions: `fix: restore corrupted journal-based-coordinator-self-compaction.md` design doc, `fix: properly restore corrupted design doc from e4a8a8d2`.
- **June 2** — fix: `harden service recovery after corrupt runtime state` (`fix-service-recovery`).

Read in sequence, these are not incidents; they are a pattern. By the first week of June, *corrupt runtime state* was a thing the system had hardened against once and would have to survive again. Nothing in the surviving record suggests the hazard was considered contagious to the task graph itself — the graph, after all, was the durable thing, and the archive behind it was append-only.

3.4 The rename — “the merger”

On **June 16**, the operator directed a wholesale rename: the Rust package `workgraph` became `worksgood`. It took five commits to land — `01726b5a` and `d4fab0a8` (partial: crate refs done, build passing), `aaa329eb` (complete, touching `src/config.rs`), `d2d5af17` (final cleanup), `664a3ada` (feat: `rename-cargo-package` (agent-5304)). In the operator’s memory this was “the merger”; in the log it is a package-and-branding rename executed by agents at 16:29 on a Tuesday.

Two days later, the graph would be gone.

3.5 The death — June 18

The only surviving witness to the death is a daemon log at the *original* checkout, `/home/bot/workgraph/.wg/service/`. Its first retained line:

```
2026-06-18T11:48:10.014Z [INFO] Coordinator tick #1927 starting (max_agents=8,
executor=codex) 2026-06-18T11:48:10.018Z [ERROR] Coordinator tick error: Failed
to load graph for task-aware reaping
```

From that tick until the log’s final entry on **June 22, 22:00**, the daemon logged the same error on every tick — **76,371 recorded failures across two rotated files** (`daemon.log.1`: 53,318; `daemon.log`: 23,053), while continuing to start coordinators with `executor=codex`. The service was alive. It was simply blind: the graph it needed to load was not there to load.

The same day, June 18, the record shows an adjacent failure in the executor layer: `fix: eliminate nex streaming interruption (error decoding response body)`, whose body documents a request-timeout misdiagnosis cutting healthy long generations — “[request] `bytes_stream()` maps EVERY body error to `Kind::Decode` ... the client’s 300s TOTAL request timeout cut healthy long generations”. Read strictly, that fix explains a streaming symptom, not the missing graph. But it establishes that June 18 was a day of compounding, partly-misdiagnosed failures in the execution layer.

What we know: the graph file was unloadable or absent at the original path by the morning of June 18, two days after the rename, in the middle of the pi migration sprint, and the failure went unobserved for at least four and a half days while the daemon ticked.

3.6 Three hypotheses, honestly stated

The operator’s hypothesis is that the failure explosion was caused *in part by switching to other models or executors that did not feel the context* of the running system — new runners acting on state they did not understand. The evidence cannot yet confirm this. Three candidate mechanisms survive the records:

1. **Corruption (the felt hypothesis).** Corruption was demonstrably recurring (§3.3). A bad save or a partial write during the churn could have destroyed `graph.jsonl` and, with it, the archive’s host directory. *For:* the history of corrupted state and corrupted docs. *Against:* no surviving log or commit names the corrupting event; the daemon’s error is “failed to load,” which is equally consistent with the file being absent.
2. **Relocation / path loss.** The original checkout’s state directory is now a husk — `stats.json` contains only `{"service": 98}`, `usage.log` is empty, and there is no `graph.jsonl`, no `archive.jsonl`, no `agents/`. That signature is as consistent with *the state having moved* (and a stale daemon looking at the old path) as with corruption. The July 4 cutover then recreated a graph from scratch at the new checkout (`/home/bot/wg/.wg`), whose earliest files are dated July 4 — so even if the state moved, its history did not survive the relaunch.
3. **Wipe during migration churn.** Multiple executors (`nex`, `opencode`, `codex`) and routing profiles coexisted in this window; the migration era was, in aggregate, a period of aggressive self-modification. An unwise migration step could have cleared state.

What would distinguish them: a filesystem-level forensic (inode history, extents, or backups) that no longer exists, or a surviving copy of the pre-June-18 graph, which the exhaustive search of this machine did not find. The honest formulation for the book: *the graph died on or before June 18, during the executor-migration era, with corruption a demonstrated recurring hazard, relocation a live alternative, and no surviving artifact that discriminates between them.*

One thing the hypotheses share is more important than which is true: **the state was durable but local, and nothing — human or agent — was watching it during the quiet.** The daemon’s 76,371 failures were logged and read by no one. The operator learned days later, in passing.

3.7 The world outside kept moving

June is also the month the project became genuinely public. Eleven Windows-portability PRs from an outside contributor (`njt`) merged — `bash resolver`, `service start timeout`, `wrapper paths`, `home lookup`, `wg doctor` — plus a multi-bot Telegram feature from a second (`lucapinello`). On **June 26**, the diverged lineages were reconciled:

Synthesize diverged mains: merge public origin/main into the internal main — “*Key fact: this merge is CONTENT-NEUTRAL... There were ZERO conflicts to resolve. The merge commit exists only to join origin/main’s history under local’s so a git push origin main fast-forwards.*”

So the “one merger” that the operator half-remembers has two faces: the June 16 package rename, and the June 26 history reconciliation. Neither destroyed content. The content that died was not in git at all.

3.8 What was lost in the quiet

By June’s end the organization had lost: the task graph for January–June (its audit chains, statuses, and history), the durable task archive that had been accumulating since spring, and any record of which agents did what in the spring. The git history survived — it lived on GitHub. The state did not — it lived only in a local directory that stopped being readable on June 18 and was superseded by a fresh state directory on July 4.

That asymmetry — remote git, local state — is the whole lesson of the chapter. The archive’s design was sound (append-only, crash-safe, restorable). Its *placement* was the vulnerability: durable, local, unwatched, in the exact directory that a rename, a migration, and four days of inattention passed through.

Timeline

- **May 1–31** — Quiet: 128 commits; operator running two grant campaigns elsewhere; `nex/opencode` executor experiments begin; corruption fixes (`nex cursor`, May 2).
 - **June 2** — `harden service recovery after corrupt runtime state` — corruption is a hardened-against hazard.
 - **June 11–14** — `opencode` executor researched and exposed over OpenRouter.
 - **June 16, 16:29** — `rename-cargo-package: workgraph -> worksgood` completes (5 commits; “the merger”).
 - **June 18, 11:48** — daemon at `/home/bot/workgraph/.wg` begins failing every tick: “*Failed to load graph*”; same day, `nex streaming-timeout` misdiagnosis fixed.
 - **June 22, 22:00** — final daemon entry (tick continuing to fail); pi migration sprint underway (`pi-plugin-research - verdict`).
 - **June 26** — diverged mains reconciled, content-neutral, zero conflicts.
-

Chapter 4 — Ignition and Cutover

2026-07-01 to 2026-08-05. 457 commits. The month the rulebook grew ten times faster, the organization moved house eight times, and the machinery began writing about itself.

The empty days

The month opens with nothing. July 1 and July 2, 2026 have zero commits in `main` — and June 30 has none either, making a three-day silence after a June 29 with a single commit. The last commits of June had been ordinary: `merge-clean-luca`, `latex-ingest`, `fix-post-publish`. Then the repository went quiet, and when it spoke again on July 3 it was with a new vocabulary.

The last three commits of July 3 read: `feat: fix-wedge (agent-5901)`, `feat: fix-agency-flip (agent-5907)`, `feat: fix-reviewer-native (agent-5902)`. These are the last commits bearing the four-digit agent numbers of the old era. From July 4 onward, agent IDs restart from single digits (`agent-1`, `agent-4`, `agent-8`). In the machinery’s own bookkeeping, the organization had been renumbered. Something had been rebuilt.

July 4: the graph is born again, and the routes open

The `.wg` directory of the quarantine record — the atomic relocation of August 5 that preserved this period — contains files and nothing earlier. The evidence stated plainly: the graph directory as it exists in the surviving record was **created on July 4, 2026**. Its earliest artifact is `config.toml.2026-07-04T15-31-36Z`, backup of a config that already carried the full `[agency]` automation block:

```
auto_assign = true
auto_evaluate = true
flip_enabled = true
auto_rescue_on_eval_fail = true
gate_confidence_threshold = 0.7
gate_max_attempts = 2
```

and, above those flags, four agent-identity hashes: `assigner_agent`, `creator_agent`, `evaluator_agent`, `evolver_agent` — the machinery’s named offices, each a SHA-256 keyed to a role.

Three hours later, at 17:48, the same directory held a migrated profile in which the offices were gone and the automation had been switched off:

```
auto_assign = false
auto_evaluate = false
```

with a new executor line beneath it:

```
executor = "pi"
model = "pi:openrouter/z-ai/glm-5.2"
```

And at 19:55 the *global* configuration layer — `~/wg`, the user-level state that survives project resets — jumped from 376 bytes to 3,376 bytes, gaining the same `[agency]` block with `auto_assign = true`, `auto_evaluate = true`, `flip_enabled = true`.

Uncertainty flagged: the global file at 19:55 is named `config.toml.bak-...`, and `wg`'s backup naming is not consistent enough to resolve from filenames alone whether that file preserves the state *before* the 19:55 write or *after* it. What is unambiguous is the clustering: within four hours on July 4, the graph was recreated, the executor was switched to `pi` with an OpenRouter model, the agency automation block was present in the global layer with assignment and evaluation both enabled, and the agent population was renumbered. Whatever the exact direction of each edit, July 4 is the day the organization changed hands from Claude to Pi and from silence to a route schedule.

The churn week: eight routes in eight days

The global config backups preserve the route history exactly. Reading the `[agent]` `model` line in chronological order:

Timestamp	Route
2026-06-23	claude:opus
2026-07-04 17:08	claude:opus
2026-07-04 17:09	codex:gpt-5.5
2026-07-04 19:55	codex:gpt-5.5 (agency block on)
2026-07-06 14:47	pi:openrouter:z-ai/glm-5.2
2026-07-07 11:31	pi:lunaroute:glm-5.2-nvfp4
2026-07-07 11:44	codex:gpt-5.5 (reverted)
2026-07-08 09:20	pi:lunaroute:glm-5.2-nvfp4
2026-07-08 11:53	pi:openrouter:deepseek/deepseek-v4-flash
2026-07-11 17:59	pi:openrouter:minimax/minimax-m2.7 and openai-codex:gpt-5.6-sol
2026-07-12 10:05	pi:openai-codex:gpt-5.6-sol
2026-07-25 11:22	pi:zai:glm-5.2

From July 4 to July 11: **eight distinct route states in eight days**. The route moved from Claude to Codex to GLM 5.2 through OpenRouter, to a NVFP4 quantized GLM on LunarRoute, *back* to Codex — a full reversal — forward again to LunarRoute, across to DeepSeek v4 Flash, and finally to Minimax m2.7 and GPT-5.6 Sol. The commit record names each move: `integrate-openrouter-login` (Jul 4), `bug-profile-use-preserve-openrouter-endpoint` (Jul 5), `value-aware model-scout with canonical handler-first weak route` (Jul 5), `pass pi custom provider models through` (Jul 7), `probe-codex-gpt-5-6` (Jul 11), `research-gpt-5-6-codex` (Jul 11), `add-direct-codex` (Jul 12), `align Codex profile assertions with Sol and Luna` (Jul 13).

The organization was not merely adding a provider. It was *shopping* — scouting models, probing new releases, comparing tiers — while running production work through whatever route was live that afternoon.

The doomed pair

What this cost is documented in a triage from later in the period, `WG_EXECUTOR_MODEL_MISMATCH_TRIAGE.md`. The failure chain it names:

“the selected model is codex-class, observed as `gpt-5.5`; the spawn path keeps or resolves the executor as `claude`; the task log shows a doomed pair: `--executor claude --model gpt-5.5`; the agent exits within roughly five seconds before doing work or calling `wg done`”

An agent launched as Claude with a GPT model name against it dies in five seconds. The root cause the triage identifies is split model/provider handling: `resolve_model_for_role` can return a bare registry model id such as `gpt-5.5` while carrying the provider separately, and bare model strings are then interpreted as Claude-compatible aliases. Prefix the model, and the executor resolves correctly; leave it bare, and the agent is launched with a mismatched harness.

This is the mechanism of the operator’s later recollection — that new models “didn’t feel the context.” The models did not misunderstand the work; the harness launched them under the wrong executor. The triage itself notes that the historical task records it needed (`causal-test-flip`, `texaudit-s8-fill`) were no longer in the local graph.

The rulebook grows ten times faster

Against that background, the constraint census recorded its spike. Constraint births by month: **5 in May, 4 in June, 43 in July**. An eight- to ten-fold acceleration, concentrated in the weeks after the routes opened.

What were the July rules about? Not, as one might expect, provider failures. Reading the 43 July-born topics:

- **r8 / pr #56, in four rounds** — Jul 10, Jul 11 (rd2), Jul 15 (rd2), Jul 16 (rd3): a scope guard forbidding durable `wg add` from a disposable scope, default-deny, then hardened per review round. Commits: feat: `--scope disposable guard on wg add (R8)`, fix(scope-guard): `default-deny durable wg add from disposable scope (R8, PR #56)`, ... rd2, ... rd3.
- **Security escapes closed in review rounds** — Jul 16: block `ls-remote ext:: remote-helper transport escape (pr-57-round-3-erik-3)`, close `shell-execution escape + guarantee timeout descendant cleanup (pr-57-round-3-erik)`, Jul 18: replace `gh deny-three` with `per-subcommand read-only option allowlist (pr-57-round-4-erik)`.
- **Review-round contracts** — never let negative-framing weaken the explicit `## Deliverables` contract (pr54-round3), route inbound replies with the `matches_sender` identity contract (PR #51 round 3), close PR 49 listener-contract gaps + confirmation race (pr-49-round-2-erik).
- **Flake fences** — `retrigger-session-lock-foreign`, flaky upstream `session_lock` pid-reuse test, deterministic `session_lock` exec-wait.
- **Branch reconciliation** — `reconcile-local-main-origin` (Jul 13), `merge-pull-request-lucapinello` (Jul 16).
- **Late-July machinery**: `flip-first-required-gate` (Jul 28), `fence-continuation-process-epochs`, `implement-lazy-evaluation`, `bound-worktree-observer-churn`, `quality-pass-dead` (Jul 26), `reframe-supervisor-as-convergence-reconciler` (Jul 31), `restore-attended-repository-authority` (Jul 31).

The mid-July pattern is distinct from the provider-contract pattern the paper describes for August. **July’s ratchet turned on review rounds and external contributors**. A pull request arrived, a reviewer found a gap, a guard was added, the guard was reviewed, a narrower guard was added, the narrower guard was reviewed again. `r8`, `pr #56` produced three hardening rounds; `pr-57` reached round four. Thirty-one commits in the window reference pull requests or the external contributor’s handle.

Caveat on the count

35% of July’s 43 constraint topics are clustering artifacts, not semantic rules — nine are named for agent IDs (agent-400, agent-664, agent-896...) and six are `rescue-checkpoint-*` markers. The honest July figure is closer to **28 identifiable constraint topics**, still a seven-fold rise over May–June. The direction is robust; the precise multiplier is an upper bound.

The machinery begins writing about itself

The daily commit histogram tells the month’s shape:

Jul 03	5 ##	Jul 16	14 ####	Jul 26	34 #####
Jul 04	6 ##	Jul 17	13 ####	Jul 27	13 ####
Jul 05	10 ###	Jul 18	19 #####	Jul 28	34 #####
Jul 06	5 ##	Jul 19	13 ####	Jul 29	6 ##
Jul 07	5 ##	Jul 20	5 ##	Jul 30	3 #
Jul 08	1 #	Jul 21	5 ##	Jul 31	33 #####
Jul 09	3 #	Jul 22	9 ###	Aug 01	8 ##
Jul 10	18 #####	Jul 23	3 #	Aug 02	18 #####
Jul 11	16 #####	Jul 24	6 ##	Aug 03	32 #####
Jul 12	8 ##	Jul 25	24 #####	Aug 04	6 ##
Jul 13	16 #####			Aug 05	33 #####
Jul 14	8 ##				
Jul 15	6 ##				

Three regimes: a slow start (Jul 3–9, single digits), a sustained review-and-harden plateau (Jul 10–19, 6–19/day), a lull (Jul 20–23, 3–9/day), then an explosion from July 25 onward — **24, 34, 13, 34, 6, 3, 33** in the last week of July, sustained through the first week of August at 8, 18, 32, 6, 33, 19.

The explosion has a signature. In the whole window, **47 commits begin with the string `wg rescue checkpoint`** and **21 begin with `wg merge candidate`** — 68 commits, 15% of the month, that are the coordination machinery narrating its own recovery attempts:

```
wg rescue checkpoint design-strong-agent generation 0 attempt attempt-0-6
wg rescue checkpoint atomic-save-kernel-schema generation 0 attempt attempt-0-1
wg merge candidate wgcid:v1:blake3:ba876c02... for fix-lifecycle-retry-watchdog-fixture
```

The first rescue checkpoint in the window appears on July 26. From that date the machinery’s bookkeeping commits — attempts, generations, merge-candidate digests, rescue markers — become a visible fraction of all repository activity. The organization was now spending a measurable share of its commit budget describing its own difficulty.

The names in that final week are the archaeology of a system trying to heal itself: `design-atomic-graph-work-save`, `atomic-save-kernel-schema`, `atomic-save-legacy-migration`, `atomic-save-worksave-capture`, `atomic-save-daemon-convergence`, `atomic-save-formal-rust-traces`, `atomic-save-terminal-adapters`, `formalize-lifecycle-finish-lean4`, `fix-exited-worker-finish-convergence-deadlock`, `reframe-supervisor-as-convergence-cutover-runtime-kernel`, `convergence-cutover-dispatch-route`. Atomic saves. Formal models in Lean. Convergence kernels. The organization built a formal specification of its own completion protocol inside the month it was failing to complete work.

July 25 – August 5: the rescue era

The last twelve days of the window are dominated by the field’s own vocabulary of emergency: *rescue*, *checkpoint*, *convergence*, *cutover*, *quarantine*, *reopen*, *fence*, *epoch*.

The lifecycle ledger preserved in the quarantine spans exactly this period — 1,475 events from **2026-07-28 09:56 to 2026-08-05 09:37**. Its most frequent event kinds are `attempt-reserved` (361), `reservation-cancelled` (189), `attempt-running` (172), `pi-continuation-authorized` (165), `reopen-requested` (121), `reopen-owner-released` (121), `pi-continuation-epoch-reserved` (115), `pi-process-epoch-exited` (64). Reservation, cancellation, reopening, continuation authorization: the ledger reads like a queue under strain, constantly cancelling and re-reserving.

By the end, this graph held 824 tasks, 1,319 attempt records, 482 completion records, 382 messages, and 209 logged agent runs.

Then the disk filled.

`graph/recovery/dispatch-cutover-disk-exhaustion-20260805T0654Z/README` states it in one line:

“Recovered 19-file WIP at commit `cb8c14e3` after `ENOSPC`; cleanup marker copied from retained worktree and disarmed to prevent evidence deletion.”

Commit `cb8c14e3` is `wip: preserve dispatch cutover after disk exhaustion` — an uncommitted 19-file work-in-progress, rescued from a worktree after the filesystem said no. The `cleanup-pending.marker` was present in the recovery directory, empty, its deletion disarmed to keep the evidence.

Also in the untracked file list at the moment of quarantine: `BUG-pi-custom-provider-model-rejected.md`, `BUG-worktree-sweep-destroys-uncommitted-work.md`, `BUG-wrapper-auto-marks-done-on-exit-bypassing-smoke-gat`, `WG_EXECUTOR_MODEL_MISMATCH.md`. Three bug reports and a model-mismatch dossier, uncommitted, sitting at the root of the repository on the day the graph was taken away.

The cutover and the quarantine

The final commits of the window are a single coordinated act. On August 5, `main` receives:

```
feat: complete worker-owned completion cutover
refactor: retire legacy completion mutators
refactor: retire automatic finalization replay
refactor: derive done from review and publication
feat: worker-owned universal completion cutover (#61)
feat: add manifest-bound FLIP and eval valve
formal: add simple universal review landing kernel
```

feat: worker-owned universal completion cutover (#61) — commit `4aca437f` — is the revision the quarantine names as its *accepted code revision*. The same commit is the target of the recovery tag.

At **2026-08-05 18:41:51Z**, the entire graph directory was relocated atomically out of `/home/bot/wg/.wg` into quarantine. The relocation left behind a single file, `DO_NOT_RUN.txt`:

“Quarantined damaged historical WG graph. Relocated atomically from `/home/bot/wg/.wg` at 20260805T184151Z. Do not start a service against this graph or migrate it in place. Preserve its task state, finalization/planner traces, recovery records, logs, and links to the historical evidence-bearing worktrees under `/home/bot/wg/.wg-worktrees`. Accepted code revision: `4aca437f135721d17fec977dff017f1f3481797` Recovery tag: `recovery/overengineered-cutover-20260805`”

Five instructions and two identifiers. The system named its own condition in the recovery tag: `recovery/overengineered-cutover-20260805`. Not *failed*, not *damaged* — although the first line says *damaged*. *Overengineered*. The organization filed its own month under that word, in a git tag, on purpose, as the label for the state from which it intended to recover.

What the month was

July 2026 was the month the organization moved to a new execution substrate and a new provider, and never stopped moving. Eight routes in eight days. A graph recreated on July 4 with agency automation switched on at the global layer. A constraint birth rate eight to ten times the previous months — driven not by provider contracts (those come in August) but by review rounds on outside pull requests and by flake-hardening. An external contributor’s four PRs producing at least four distinct hardening rounds. Then, from July 25, an acute rescue phase: 15% of all commits in the month narrating the machinery’s own recovery attempts, atomic-save machinery built to stop the graph corrupting, a formal Lean specification of the completion protocol written while completion kept failing, and finally a disk that filled, a work-in-progress rescued from a doomed worktree, and a whole graph carried out of its home at 18:41 on August 5.

The month ends with the graph in quarantine, the ledger preserved to the byte, and a recovery tag that reads like a diagnosis.

Timeline

1. **Jul 1–2** — zero commits (the month’s only silence); Jul 3 last commits under the old four-digit agent numbering.
 2. **Jul 4** — graph recreated (~15:31); `[agency]` automation block present with `auto_assign/auto_evaluate/flip_enabled` true; executor switched to `pi` + OpenRouter GLM 5.2 (17:48); global config gains the agency block (19:55).
 3. **Jul 4–11** — eight distinct route states in eight days (Claude Codex GLM 5.2/OpenRouter GLM 5.2-NVFP4/LunarRoute Codex revert LunarRoute DeepSeek v4 Flash Minimax m2.7 + GPT-5.6 Sol).
 4. **Jul 10–19** — review-round hardening peak: `r8/pr #56` in three rounds, `pr-57` to round four, security-escape closures, session-lock flake fences; 43 constraint topics born in July (28 after excluding clustering artifacts) against 5 in May and 4 in June.
 5. **Jul 25 – Aug 5** — rescue era: 47 `wg rescue checkpoint` + 21 `wg merge candidate` commits (15% of the month); atomic-save and Lean-convergence machinery built; ENOSPC disk exhaustion on Aug 5 with a 19-file WIP recovered at `cb8c14e3`; `worker-owned universal completion cutover` (#61) lands; the graph is atomically quarantined at 18:41:51Z under the recovery tag `recovery/overengineered-cutover-20260805`.
-

Corrections and uncertainties carried forward

- **The provider-backoff-contract was not born in July.** Its five commits are dated 2026-08-16 and 2026-09-09. Any narrative placing its birth in the July churn week is unsupported by the commit record; July’s provider work was route selection (`integrate-openrouter-login`, `pass pi custom provider models through`, `probe-codex-gpt-5-6`), not backoff contracts.
 - **The direction of the 19:55 global config write is unresolvable** from filenames alone; the agency block is present-and-enabled in the file bearing that timestamp, and the project-level config shows the same flags toggling within the same afternoon.
 - **One third of July’s constraint topics are clustering artifacts** (agent-ID and rescue-checkpoint labels), so 43 is an upper bound and ~28 is the defensible count.
 - **The `[agency]` block appears at three layers on July 4 with different settings** (project config 15:31 enabled with agent-identity hashes; migrated profile 17:48 disabled; global 19:55 enabled). The record does not make the precedence between these layers explicit.
-

Chapter 5 — Reset and Rescue (August 5–12, 2026)

The week the organization was relocated, re-cloned, re-founded — and audited itself to a constitution.

By August 5, the machinery had eaten the graph. The coordination state that had been quarantined that morning — 824 tasks, the accreted rulebook, months of attempt records — was damaged beyond in-place repair, its own recovery directory littered with the week’s failures: `dispatch-cutover-disk-exhaustion`, `terminal-rewrite-loop`, `planner-schema5-bootstrap`. Disk exhaustion, save corruption, planner bootstrap failures — the substrate itself was failing, and the governance layer had grown too dense to survive on it. The quarantine note said what it was: a damaged historical graph, relocated atomically, do not run.

What followed was not one rescue but three, layered over four days: a teardown, an audit, and a refounding — each with its own record.

The teardown day

August 7 is the densest deletion in the repository’s history. The commit subjects from that single day read like a demolition order executed by a crew that knew exactly which walls to drop:

- “Delete evaluator reconciliation status authority”
- “Delete completion-side validation authorities”
- “Quarantine legacy completion through lifecycle”
- “Delete evaluator-driven retry authority”
- “Remove dormant evaluator repair controller”
- “Retire legacy FLIP routing integration tests”
- “Merge simplified lifecycle control plane”

Forty-seven commits landed on August 7 alone (159 across August 5–7), and their verbs are overwhelmingly subtractive — *delete*, *retire*, *quarantine*, *remove*, *drop*, *prune*. This was the enforcement-autonomy disable of September 13 in its first, cruder form: days before the config-level change the recovery narrative remembers, the organization was already tearing out the machinery that added and enforced gates without a human decision. The “simplified lifecycle control plane” that replaced it is the ancestor of everything that works today.

The audit paradox

Then, in the middle of the teardown, the organization did something that looks contradictory until you understand what it had learned: it audited itself. Between August 7 and 9, 76 commits carry “audit” in their subject. The result was a fourteen-part system audit — code architecture, orchestration lifecycle, model execution plane, agency-evaluation, federation and security, documentation, testing — plus a synthesis, an independent review of the synthesis, a calibration of the review’s evidence, and a “cross-audit contradiction, drift, and uncertainty register.”

The register is a remarkable document. Written by agents about their own organization, dated August 8, it deduplicates “repeated IDs for the same underlying conflict” across fourteen audits and records, in its own labels, the drift that had accumulated: `wg done --help` still advertising five legacy flags while the code rejected them; review/evaluation prose describing “ratified or complete enforcement” where the implementation “provides a narrower spark”; point-in-time designs “preserved correctly but lack status, applicability, and supersession edges.” The organization had learned to write its own contradiction register — the exact artifact a bureaucracy needs and almost never produces.

The rescue manifesto

On August 9 at 08:00, the operator committed the document that anchors the whole week: “**Simple Local WG Recovery Work Document**” (357 lines, branch `rescue/simple-local-wg`, status “ACTIVE —

operator recovery; WG dispatch is stopped”). It is the clearest statement in the entire record of what the system had become and what it was supposed to be.

The mission section restores, in seven numbered lines, the original contract: “A user asks for work. Chat creates a task graph. Trusted local workers fan out and coordinate freely through that graph... Tasks become Done without hidden controller deadlocks or manual object-store surgery.” And it names the intervention precisely: “This is a **semantic rollback**, not a blind historical Git revert” — keep worktree isolation, attribution, fencing, immutable evidence, accounting; remove “default permission and completion machinery that prevents trusted local work.”

Then the evidence section lists, with dates and counts, the disease as observed during the August 8–9 audit run:

- “The trust-first fix passed its substantive validation, then accumulated at least 10 consecutive FLIP rejections,” expanding to 15 commits and ~3,653 inserted lines “including new ‘seal’ and ‘fail closed’ controls contrary to the requested simplification.”
- “The build-admission fix initially passed build, format, clippy, targeted tests, install, and owned smokes, then accumulated at least 20 review rejections and expanded to 28 commits and approximately 1,628 inserted lines.”
- A malformed capability path “prevented a dead worker from even recording its own failure.”
- The documented escape hatch `wg done --skip-verify --skip-smoke` was itself “rejected with legacy `wg done bypass/merge/cycle` flags are not supported by publication-derived completion.”

The sentence that summarizes the whole chapter is the document’s own diagnosis: **“These are control-plane failures, not failures of the requested source, audit, or scientific work.”** The accretion was fighting the rescue in real time — the requested *simplification* was being expanded by review machinery into new seals and fail-closed controls, exactly as the census predicted a rulebook under incident pressure would behave. A “non-negotiable target contract” section then deletes by name: no model reviewer “with unbounded source-task blocking authority,” no mandatory quality-pass dependency, no hidden build concurrency limits, no infrastructure outage classified as scientific failure.

The unmerged survivors

The rescue produced work that never fully reached main. A scan of all 56 local branches finds seven with commits main does not contain: `wg/agent-23/restore-trust-first-local-worker-control` (15 commits), `wg/agent-24/fix-build-admission-default-ui` (28), `wg/agent-41/completion-classification-precedence` (23), `wg/agent-43/completion-review-candidate-budget` (13), and three smaller ones — 87 unmerged commits in total, all from the rescue fortnight. The August 9 recovery bundle preserves them (`preserved-refs.bundle`, with `rescue/simple-local-wg` and `main` pointing at the same commit, `e12ee37c`), which is why this chapter can be written at all: the rescue was reflog-fragile — the reflog shows repeated `reset: moving to ...` entries through August 9–10, including three in the early hours of August 10 — and the bundle is what made the branch state durable.

Whether the unmerged 87 were superseded, abandoned, or simply left behind is not determinable from the refs alone; they are flagged here as the record’s one open drawer.

The new order

By August 10, the replacement order was writing itself down. “Receipt-backed scored evaluation” defines what an evaluator is now allowed to be: “deliberately narrower than task completion,” observation and findings only, one bounded call, “no task status, lifecycle, retry, publication, or graph node can be changed” — evaluators demoted from authorities to witnesses. The same day, “record stigmergic loop validation” closes the loop on the term this book has used throughout: the organization validated that its coordination now worked by stigmergy — traces in the environment, not commands in a hierarchy.

The cast for the week: agents 21–43 and the audit fleet; the operator, present in every rescue document as “repository operator”; and the daemon, finally loading a graph it could serve.

Timeline

- **Aug 5, 18:41** — damaged graph atomically quarantined (“overengineered-cutover”); disk-exhaustion recovery capture same day
- **Aug 7** — 47-commit teardown day: evaluator authorities deleted, FLIP routing retired, simplified lifecycle control plane merged
- **Aug 8** — fourteen-part system audit + contradiction register published; **wg done** escape hatch found rejected by the machinery
- **Aug 9, 08:00** — operator’s rescue manifesto committed; semantic rollback declared; 76 audit-adjacent commits total across the run
- **Aug 10** — scored evaluation demoted to observation-only; stigmergic loop validated; repeated reflog resets end; the new order holds

Uncertainties flagged: individual commit authorship within the rescue (operator versus agents) is not distinguishable from commit metadata alone; the disposition of the 87 unmerged branch commits is unknown; the precise cause of the August 8 reflog resets (three before 03:00) is unrecorded.

Chapter 6 — Recovery and Burden: The Gummy Month

August 12 – September 13, 2026

(commits 2026-08-12 2026-09-13; ledger window Aug 8–Sep 16; 99 commits in window)

The silence after the quarantine

The gummy month begins in near-silence. After the twin traumas of early August — the atomic quarantine of August 5 and the re-clone from GitHub on August 7 — the primary organization produced almost nothing for three weeks. The commit record shows: four commits on August 16 (the provider-backoff-contract docs burst, closing its three-week-old ambiguity gaps), one on August 20 (a completion-budget fix), then nothing until September 2. Nineteen days of near-total quiescence, August 21 through September 1.

The ledger agrees. Of 115 distinct tasks in the post-reset ledger, August’s share is thin: 61 attempts reserved, 39 unique tasks succeeded, 12 attempt-failure events, 5 abandonments. The organization was alive but moving slowly — a convalescent pulse after the cutover.

The configuration tells you why. The surviving pre-intervention snapshot (`config.toml.bak-2026-09-13T02-32-08Z`) shows the machinery in its wounded state: `auto_evaluate = true` still on, but `flip_enabled = false`, `auto_assign = false`, `auto_create = false`, `auto_triage = false`, `auto_rescue_on_eval_fail = true`. The automation that had compounded the spring’s cascade was off; what remained was a system running on attended, half-disabled machinery — capable of work, but with every movement requiring supervision. The deferral record shows the cost of that posture: even in quiet August, tasks were being held at the door (2 deferral events, an early tremor of what September would bring).

The deepening tightness

The task mix in late August and early September tells a story of an organization working on its own reliability rather than its purpose. The commits are dominated by governance-adjacent repair: the provider-backoff-contract docs trio (Aug 16), a completion-rejection budget scoped to source attempts (Aug 20), then — after the September 2 resumability fix and the September 3 design document for project-local Pi configuration — a rapid sequence of authority hardening: “reject all global routing aliases” and “finish project-local Pi authority contract” (both September 6), “bound completion review evidence in time,” “pin causal completion

review boundary” (September 6), the genuine-FLIP-v2-seed pair (September 5), “Make completion rejections and landing leases authoritative” (September 5).

This is the gummy pattern in commit form: the machinery of acceptance — landing leases, review boundaries, evidence binding, canary gates — receiving obsessive attention while the product itself waits. The commit vocabulary of the window is telling: *bound, pin, scope, exact, authority, boundary, fence, gate, lease, canary*. The org had learned from its collapse that discipline was survival, and it was applying discipline everywhere — including where it would soon become “hardcore and gummy.”

September 4: the fleet re-routes

Amid the tightening, one date marks a quiet revolution: September 4, when two config commits routed Pi workers through “Codex Sol” and Pi agency reviews through “Codex Luna,” followed by “make project config sole execution authority.” This is the gpt-6-astra-era fleet arriving — the mid-September model-generation shift the metering data confirms (gpt-6-astra: \$73.61 across 329 billed messages after this date). The organization was re-founding its model stack on project-local Pi authority at the same moment it was tightening every acceptance surface. New models, new authority structure, new tightness — all in the same week. The coincidence is sharp: the model-generation shift and the peak tightness arrive together, and the recovery that follows will be credited to the config change. The ledger cannot fully separate these causes; the paper says so.

The deferral anomaly

Then September 3–4 opened the gates of a strange new phenomenon: admission deferrals. The ledger records 2,505 admission-deferred events in September against 2 in all of August — clustered at Sep 3 (789), Sep 15 (1,567), Sep 16 (112). The deferrals continued and even peaked *after* the September 13 intervention, so they are not the cost the intervention removed; they are the signature of an admission filter that stayed active throughout, through which a resurgent task population now had to pass. The task mix suggests the source: a refresh-build-baseline-admission task repeatedly re-attempting admission. The org had, in effect, an admission bouncer that became the busiest worker in the building.

The Sept-13 boundary measured at task grain

The ledger’s sharpest measurement brackets the intervention precisely:

7-day window	attempts	unique succeeded	failed	abandoned
Sep 6–12 (before)	14	9	5	0
Sep 13–19 (after)	51	35	6	4

Unique completions rose 3.9×; attempt volume rose 3.6×; the failure rate fell from 36% to 13% — not because fewer attempts failed in absolute terms (5 6), but because far more attempts now succeeded. The system resumed *attempting* work at scale the moment enforcement autonomy was disabled. The after-window’s task mix is recovery-and-release work: **first-user-release-rehearsal**, **fix-first-user-landing-ux**, **unify-project-route-authority**, repeated **route-verification-probe** runs — the org repairing its front door before opening it.

The audit-charter receipt

The window’s defining artifact is the surviving completion transaction for **audit-charter** — all five phases (implementation, validation, manifest, FLIP, evaluation) succeeded, and publication was then refused by three independent control surfaces: land refusing overwrite, the control plane refusing an unmerged-worktree override, and the waiter refusing on missing park attestations. Every gate passed; the task died anyway. This is the gummy month’s thesis in a single receipt: acceptance had become over-determined, and correctness no longer implied completion.

Reconciling the two stories

The commit record and the ledger disagree in emphasis, and both are true. Commits say: quiet August, then an explosion of acceptance-hardening and route-authority work September 4–13, with the intervention itself nearly invisible in git (a config-layer event) and the following days dominated by baseline fences and admission repairs. The ledger says: quiet August, a task-starved system that resumed attempting work at $3.6\times$ volume the day autonomy was disabled, with completions rising $3.9\times$ and the failure rate halving. Together: the gummy month was real (deferrals, tightness, the receipt), and the intervention worked on the dimension the commits couldn't show — the permission to attempt.

Timeline

- **Aug 16** — provider-backoff-contract docs trio; ambiguity gaps closed, no functionality shipped
- **Aug 20 – Sep 1** — near-silence: one commit in twelve days; machinery half-disabled, org convalescing
- **Sep 2–6** — project-local Pi configuration designed, migrated, proven; routing aliases rejected; authority contracts bound
- **Sep 4** — fleet re-routes to Codex Sol/Codex Luna (the gpt-6-astra arrival); project config made sole execution authority
- **Sep 13** — enforcement autonomy disabled; completions 935/week; baseline fences and admission repairs follow immediately

Chapter 7 — Resolution and Constitution (September 13–18, 2026)

The Book of WG, chapter 7. Sources: git log on main (2026-09-13 through HEAD), the .wg/config.toml.bak-2026-09-13T02-snapshot, the current live configuration, the coordinator chat record, and docs/reliable-work-guiding-principles.md (b025694e).

The night of September 13

The intervention began at 04:32 local time — the configuration snapshot `config.toml.bak-2026-09-13T02-32-08Z` was written at 02:32 UTC, which is 04:32 in the operator's timezone. The commit record knows nothing of this: the change was made entirely in the configuration layer, where git does not reach. What the config snapshot preserves is the moment the [agency] automation-and-gate block was demobilized — `auto_evaluate` and the autonomous evaluation loop switched off, `auto_assign` and `auto_create` disabled. The machinery that had been adding and enforcing constraints without a human decision was, from that moment, unable to act unbidden.

The coordinator chat record preserves the operator's contemporaneous voice from these hours: “jfc *FLIP is being an asshole... these hardcore checks are hardcore and gummy. we can't work reliably.*” This is what rule bankruptcy looks like from the inside: late-night, exasperated, and entirely hand-performed.

The morning after

At 08:54 on September 13, the first commit lands: `fix: refresh exact cargo baseline admission (fc2c9ddd)`. Then a tight sequence over the following hour:

- 09:13 — `test: exercise production baseline publication (01e33c07)`
- 09:22 — `fix: scope cold baseline fences by exact key (1bcded95)`
- 09:37 — `test: cover baseline failure recovery matrix (a925f85f)`
- 09:41 — `docs: record baseline admission reproduction (5a0b28a6)`

This is the recovery made visible at task grain: the seven-day window before the configuration change contained 9 unique task completions; the seven days after contained 35 — a $3.9\times$ rise, with attempts rising

from 14 to 51 and the failure rate falling from 36% to 13%. The morning-after commits are the repair of the *measurement apparatus itself*: baseline admission, baseline publication, the failure-recovery matrix. Before the organization could work again, it had to be able to see what “working” meant.

That afternoon brought two quieter commits — `Allow completion help after semantic rejection` (caee4aed) and `Persist completion repair saved-work locator` (cfeab43c) — both softening the acceptance edges that had been strangling completion: work that had been semantically rejected could now get help instead of dying.

September 14: the constitution

The next day at 16:58, the organization wrote down how it intended to work. Commit b025694e — `docs: define reliable-work operating principles` — added 245 lines to `docs/reliable-work-guiding-principles.md`. It opens with what the document calls **the promise**:

“Give WG a bounded piece of work. Receive an inspectable result or a precise request for a necessary decision. Work does not disappear, and the user does not have to repair the coordination machinery.”

And then, the sentence that reads as a direct verdict on the preceding five months:

“This is a work-coordination system, not a ceremony-enforcement system.”

The constitution’s first principle is “Make the ordinary path small”: *Understand the task do the work check the result repair if appropriate finish or ask for help*. It explicitly warns against the pathology the census had measured: “A run that succeeds only after repeated manual retries, contract edits, or daemon restarts is not an autonomous success, even if the task eventually says Done.” The July dispatch-inflation signature — 2.6% 25.8% 26.8% of completed tasks needing multiple dispatches — is here forbidden in principle. The document also draws the distinction the whole crisis had blurred: **coordination requests are not product acceptance criteria** — “a request for an early message must not become a requirement to prove the message’s timestamp inside the candidate manifest.”

The same evening, the repair continued: `fix: restore reliable baseline routing tests` (17:27) and `docs: record baseline failure diagnosis` (17:37).

The redirected care

The semantic analysis of the commit corpus shows what happened to the organization’s characteristic language. Guarantee-vocabulary density (never, enforce, reject, guarantee, must) ran at 85.9 per 100 commits in April — the accretion months — collapsed to 8.1 in June, and returned to 61.6 in September. But September’s words are different words: *prove, exact, baseline, reconcile*. The same energy that had spent the summer walling work in was now spent proving the substrate: stable environment identity, deterministic validation evidence, hermetic tests.

The hardening did not die. It changed object — from gating the work to proving the ground it stood on.

September 16: the great day of forward work

September 16 is the densest single day in the repository’s post-crisis history — roughly forty commits in one day, and their subjects tell a story of an organization doing something it had not done since spring: **forward feature work, without the burden signature**.

The day’s headliners:

- `feat: embed a live, read-only VizView work-graph panel in the wg-pi plugin` (a490d852) — the organization gained a live view of its own graph.
- `docs: design retiring the WG model registry in favor of Pi's models-store catalog` (277ef3ac, 460 lines) and its implementation the same night (b5019738, 23:13) — the WG model registry, a piece of in-organization governance, was **retired in favor of externalizing the function**

to the host harness. The design document is authored by Erik Garrison directly; the milestone was executed the same evening.

- **feat: concierge-guided Pi readiness gates in attended interactive setup** (cea99615) — the new pattern for gates: attended, concierge-guided, operator-present. Gates by decision, not by emergence.
- **feat: reap orphaned descendant process groups on agent death** (60de44a8) — real infrastructure repair, the kind of work that had been impossible in the burden era.
- **test: make lib suite hermetic against cross-test env races** (a5ee70ce) and **test: pin completion-pipeline fixes with three credential-free smoke scenarios** (6eb90375) — the test suite hardened against its own environment, not against its own workers.
- **docs: npm distribution decision doc for WG binaries** (54fd6424) — thoughts of shipping, of users beyond the operator. The first sustained sign in months that the roadmap existed.

The day also shows the new normal’s texture: opaque-Pi-execution trial reports with admission blocked by admin boundaries (c8928c2f, 3484fbd1 — trials run *under operator authority*, a phrase that would have been unnecessary in August), route-hardening batch synthesis with config-hygiene migration “flagged as not executed” (59d35b7f — work is now *deferred deliberately* rather than silently compounding), and a chore dropping migration backups from the repository (5e960d24 — hygiene).

The ending, and what it cost

Two honest footnotes. First, the resolution was not instant: a test fix for cross-crate module paths (**fix E0433 in bin-crate test modules**) landed on September 18, two days after the constitution’s implementation wave — the substrate was still being stabilized. Second, the cost: September’s capacity went into teardown, baseline repair, and constitution-writing rather than the roadmap. Subtraction consumed the fall — the retirement valve was exercised by hand, late, and it was expensive. The constitution’s own promise — “*the user does not have to repair the coordination machinery*” — describes a state that, as of HEAD, the organization had earned only by a month of exactly that repair.

Timeline

- **2026-09-13, 04:32 local** — enforcement autonomy disabled in configuration (invisible to git; the intervention itself)
- **2026-09-13, 08:54–09:41** — the baseline sequence: five commits repairing the measurement apparatus
- **2026-09-14, 16:58** — the constitution: 245 lines of reliable-work operating principles (b025694e)
- **2026-09-16** — the great day of forward work: model registry retired, VizView panel, concierge-guided gates, process-group reaping, npm distribution (~40 commits)
- **2026-09-18** — last stabilization fix at chapter close (E0433 test modules)

Where the org stands at HEAD: the automation is off and stays off; the registry is externalized; the constitution is written and its promise is explicit; forward work is flowing again. The system that remains is smaller than the one that cascaded — and its constraint ledger, for the first time in the organization’s history, is one that nobody is afraid to read.

Chapter 8 — The Executor Saga

Thematic chapter. The organization’s nine-month effort to decide who — or what — actually runs the work.

The tax

Every eighth commit in wg’s entire lifetime was about moving house.

By subject-line grep across all 3,226 commits, **511 commits (15.8%)** mention an executor, provider, route, profile, or model. The semantic classifier run over the same corpus scores provider/executor migration at **430 commits, 13%** — the single largest non-implementation theme in the repository, larger than verification, larger than documentation. An organization built to coordinate agents spent a seventh of itself deciding which agents to coordinate it with.

The distribution is not flat:

Phase	Window	Commits	Executor/provider-touching
Native + Claude	Jan 18 – Apr 17	1,805	424
Codex adapter	Apr 18 – May 5	481	70
Nex / OpenRouter	May 6 – Jun 9	113	37
Opencode	Jun 10 – Jun 21	66	13
Pi arrival	Jun 22 – Jul 3	184	66
Churn week	Jul 4 – Jul 25	255	72
Sol-stable	Jul 26 – Aug 31	552	144
Project-local Pi	Sep 1 – Sep 30	175	78

The first row dominates by volume because the whole organization was being built then. The interesting rows are the middle ones: in the six weeks from May 6 to June 21, **85 of 179 commits (47%)** concerned executors, providers, or routes — in a period when total volume had collapsed to a fraction of spring’s. **The organization was doing less work, and nearly half of it was about how work gets run.**

The zoo

The executor catalogue grew by accretion, each addition locally reasonable.

February 18 — feat: add amplifier as built-in executor. The first external harness joins the native executor.

April 18–20 — the external-executor abstraction becomes real: docs + config: external-executor class design + claude/codex TOML examples, then feat(phase7): wg codex-handler - third executor adapter, three-way coord smoke. Three adapters: native, Claude, Codex. A design pattern is now established — *executors are pluggable, so add executors.*

April 3 – May — OpenRouter arrives as a *provider* underneath those executors, not an executor itself. The arc is dense and fast: March 5 research OpenRouter integration status and gaps; March 7 OpenRouter provider client with attribution headers and URL fix + OpenAI SSE streaming with tool call accumulation; March 8 OpenRouter end-to-end validation report; April 3 a cluster of smoke tests (exec-smoke-tool-loop, exec-smoke-streaming, exec-smoke-errors, exec-smoke-wg-tools, and integration tests); **April 11 implement OpenRouter cost cap enforcement** — the provider is now metered, and a cap is a new kind of rule.

May 31 — the directive that opens the zoo. In the principal’s own words, recorded in the Codex prompt corpus:

“i want to enable all the coolest executors. opencode! then the next most popular ones that work on an open backend. amplifier. anything else. dispatch research”

June 1 — ten exec-* commits land in a single day: exec-opencode-aider (agent-21), exec-goose-qwen-cline (agent-11), exec-crush-amplifier (agent-12), exec-openrouter-normalize (agent-5), exec-final-integration (agent-42), exec-discovery-docs (agent-23), exec-tests (agent-22), exec-rank-defaults, exec-smoke-arena, plus fix-nex-openrouter (agent-36). Alongside them, the reliability complaints that would define the season:

“get openrouter set up correctly and smoke test nex and wg nex and opencode etc. here is our key ~/.openrouter.key. i fixed it”

“fix-nex-openrouter is stuck too pending-eval forever”

The key arrives as a file the principal fixes by hand. The executor does not work; the *route* does not work; nobody can yet say which.

June 8–14 — the zoo consolidates into *nex*, which gets its own line of development: *nex-openrouter-default-ui*, *nex-optional-openrouter-endpoint*, *research-openrouter-cli-executors*, *expose-opencode-openrouter*, and a long tail of TUI fixes (*fix-tui-opencode*, *fix-opencode-tui*, *fix-opencode-build*, *verify-opencode-chat*). By mid-June the organization is maintaining an executor that exists to test other executors.

The answer arrives as a question

June 15, 13:36 — one line in the prompt record:

“thinking we should test out and integrate the executor called ‘pi’. wdyt?”

This is the pivot of the entire saga, and it arrives as a suggestion, not a plan.

June 22 — *pi-design-integration*: *pi.dev* integration plan (executor handler + chat/TUI + light-touch patch). Same day: *pi-plugin-impl-handler*, route *pi* handler through plugin.

June 23–24 — the architectural idea that makes *Pi* survivable: *design-handler-first* (*agent-5674*), *implement-handler-first* (*agent-5701*), *fix-pi-handler* (*agent-5707*).

Handler-first is the decisive inversion. Under the accumulating design, *wg* had to *know* about models — to resolve them, register them, validate them, price them. Handler-first says: *wg* passes the route spec through verbatim (*pi:zai:glm-5.2*, *claude:opus*) and the executor resolves it. No network. No registry lookup. No central model knowledge. It is, in miniature, the resolution of the entire saga — and it was written in June, five weeks before the dispatcher actually wedged on a missing API key.

The churn week

Through June 23 the stable regime was *claude:opus* primary with *claude:haiku* for evaluator/assigner/flip roles. Then, in the week of **July 4–11**, the **primary model/route changed seven times**:

1. **Jul 4, 17:08** — *claude:opus* *codex:gpt-5.5*
2. **Jul 4, 19:55** — the [agency] automation-and-gate block switched **ON**; OpenRouter keyring adopted; `on_provider_failure = "pause"; provider_failure_threshold = 3`
3. **Jul 6, 14:47** — *pi:openrouter:z-ai/glm-5.2*
4. **Jul 7, 11:31 11:44** — *lunaroute glm-5.2-nvfp4*, reverted to *codex* thirteen minutes later
5. **Jul 8, 09:20 11:53** — second *lunaroute* attempt, abandoned same day for *deepseek-v4-flash* via OpenRouter
6. **Jul 11, 17:59** — *openai-codex:gpt-5.6-sol* (the long-stable regime begins)
7. **Jul 26, 09:59** — *pi:zai:glm-5.2* (direct *z.ai*, bypassing OpenRouter entirely)

The configuration file that held all this was **341 bytes on June 23**. On July 4 at 19:55, one edit took it to **3,376 bytes** — a ten-fold jump, because that is the moment the 25-key [agency] block landed. The project config grew from 3,376 to **5,129 bytes between July 4 and July 25: +52% in three weeks**.

The naming in that config is the saga in one line: `on_provider_failure = "pause", provider_failure_threshold = 3`, the OpenRouter keyring — all arriving **on July 4, the same moment gate automation switched on**. The infrastructure was being hardened against provider failure by the same mechanism that hardened it against everything else.

The wedge

July 25 — the commit that explains the entire problem in its own words:

```
fix(dispatch): decouple spawning from the OpenRouter catalog refresh
```

“A non-built-in provider model (e.g. `pi:zai:glm-5.2`) used to wedge the dispatcher when the OpenRouter-backed registry refresh was unavailable (no `OPENROUTER_API_KEY`): the refresh failed into a 60-min cooldown that looked like a wedged dispatcher, and the model-resolution warning made the failure look unfixable without restoring **the accidental key dependency**.”

Read that again. wg’s own model registry — the catalogue of models it was willing to route to — was refreshed from OpenRouter’s API. It had a **circuit breaker**. When the API key was absent, the refresh failed into a **sixty-minute cooldown** that presented to the operator as a wedged dispatcher. **Metadata about models had become load-bearing infrastructure, and its absence could stop all work for an hour.**

The fix is the saga’s turns in miniature: (1) decouple dispatch from the refresh — “the refresh is metadata; its absence cannot wedge dispatch”; (2) fail-soft resolution for handler-first specs; (3) first-class static registration; (4) an opt-out flag; (5) surface genuinely unresolvable models loudly at startup instead of in daemon logs.

What the saga cost

Set against the constraint census, the executor saga is the ignition’s fuel. The April credit-exhaustion events (three tasks abandoned on **HTTP 402 with minimax/minimax-m2.7**, one exit-143 agent kill) were the spring’s provider pain. July’s churn week bracketed the census’s constraint spike (43–46 births/month, against ~5 in May and June) and produced the provider-backoff-contract — five docs-only hardenings, zero shipped features. Provider failure was not a background condition of the accretion; **it was its precipitating environment**.

And in the provider domain alone, the human *did* direct hardening explicitly. **August 16:**

“Let’s implement a kind of exponential fall off, back off, when we see provider failure information.”

That is not a complaint routed around; that is a hardening directive in the realm where the principal’s expertise applied. It produced the provider-backoff-contract — which then failed, was debugged, failed again, and was hardened four more times. The one place the operator asked for a rule, the rule machinery still over-produced.

Verdict

The executor saga cost the organization roughly a seventh of its total output, a third of its commits during the summer, a wedged dispatcher, a five-week debugging arc on a contract that shipped nothing, and at least one hour of total organizational downtime per missing API key. It also directly ignited the constraint accretion that this book is about — the provider-backoff-contract family exists because endpoints failed, and it was born in the same week as the automation that let such contracts compound. What fixed it was not a better model, and not a better provider: it was **ceding authority**. Handler-first (June) made route resolution the executor’s problem; `openrouter_refresh = false` and dispatch decoupling (July 25) made provider metadata non-load-bearing; the September 3 Pi-only proposal — “*We now use Pi as the sole model/authentication interface. Provider login and credentials belong to Pi, so WG’s global secrets and model-routing state are usually unnecessary*” — and the **September 16 registry retirement** finished the job: wg deleted its own model registry, its daemon registry-refresh path, its circuit breaker, and its OpenRouter fetch, and read rates from the host harness’s catalog instead. The organization stopped trying to be a model-routing authority. One commit in eight had been spent building that authority; one commit ended it.

Uncertainties flagged: (1) The per-phase commit counts use my own date-window boundaries, which are analytical, not organizational — the phases overlap at their edges and the “native + Claude” row is inflated by the fact that all early development was executor-adjacent. (2) The 15.8% subject-grep figure and the 13% semantic-classifier figure measure different things (term presence vs. classified theme); both are reported rather than reconciled. (3) The seven-route-change count follows the config backups, which are timestamped but may omit changes reverted within a single backup interval; the true count is 7. (4) Attribution of the July

4 automation switch-on remains open — it falls in a coverage gap between the Claude (ends Jul 3), Codex (interactive ends Jul 12 and shows no matching directive), and pi (begins Aug 5) corpora.

Chapter 9 — The Machinery Biography

The life and death of the governance machinery (gates, FLIP, agency, evaluation)

Every claim in this chapter is dated from the commit record, the configuration-backup series preserved in the August 5 quarantine, or the source code at the cited revision. Uncertainties are flagged inline.

Prologue: the machinery was three organisms, not one

What the September teardown called “the governance” was, by autopsy, three distinct organisms sharing one habitat. The **Agency** — born February 6, 2026, in the commit “Implement Agency system: evolutionary agent identities for workgraph” — was an evolution engine: assigner, creator, evaluator, and evolver roles, each carrying its own blake3-hashed identity, breeding and scoring agent primitives. The **FLIP** layer — born March 3–5, 2026 (“FLIP scoring, eval-can-fail, TUI improvements”; “add FLIP-triggered Opus verification for low-scoring tasks”) — was the quality gate: it scored work and summoned Opus to verify what scored low. The **verify-gate machinery** — born in the April 5–11 burst (verify timeout raised 120s300s on April 5; “implement verify timeout triage system” and “comprehensive verify timeout functionality tests” on April 11) — was the admission control. Three organisms, three birthdays, one shared fate.

I. Infancy (February–March 2026)

The Agency arrived first, and its founding document is worth quoting: *evolutionary agent identities* — the ambition was not bookkeeping but breeding. By March 13 the evolver had a fan-in synthesizer “for evolver multi-strategy merging.” The strategy set, preserved at `src/agency/evolver.rs`, is a small museum: mutation, gap-analysis, retirement, motivation-tuning — and, excluded from the safe list because they “require human judgment,” crossover and **bizarre-ideation**. The configuration shipped a `bizarre_ideation_interval` (10) — the machinery contemplated, on a schedule, the generation of bizarre ideas. It also shipped a `ucb_exploration_constant` of 1.4142135623730951 — the square root of two: a multi-armed bandit algorithm sat inside the governance configuration, allocating exploration across the org’s own strategies.

March 3–5 brought FLIP: scoring, eval-can-fail, and the trigger that would define the next six months — *low-scoring tasks get Opus verification*. The gate now had a ratchet head: every failure could be answered with a check.

II. Adolescence (April 2026)

April was the machinery’s growth spurt, and it coincided precisely with the org’s first great pain. The April 11 triage documents credit-exhaustion abandonments (HTTP 402 on minimax-m2.7) and eight failed tasks; the gate census links seventeen constraint births to that single week, almost all of them verify-gated — `implement-verify-lint`, `inject-verify-gate-task`, `smoke-verify-gate`, `.verify-bypass-verification-for`. The hardening-language series (semantic analysis of all 3,226 commit subjects) records the same fever in vocabulary: **650 commits in April, 213 containing verify-language, 67 containing must, 52 never, 47 prove** — the densest hardening month of the organization’s life. The verify-timeout machinery grew teeth the same week (April 5: timeout doubled; April 11: a triage system for verification failures).

The detail that would matter later, visible only in hindsight: the evolver’s retirement logic — `roles_below_threshold` — exists in the source exclusively behind `#[cfg(test)]`, with a guard clause that reads like prophecy: *roles with fewer evaluations are never flagged for retirement (they haven’t been tested enough)*. In April, evaluation was still cheap and running. The guard was harmless.

III. The long silence (May–June 2026)

May and June are the machinery’s silent films. Eleven commits in June, five in May; hardening vocabulary in June: **zero** — not one guarantee, not one proof, not one enforce in the entire month. The operator was running two grant campaigns through other deployments. The machinery idled: the [agency] automation persisted in configuration, but the record shows no evolution cycles, no FLIP storms, no verify bursts.

Then June 2: “harden service recovery after **corrupt runtime state**” — the first commit to name corruption as a fact of life. June 16: the cargo package renamed from workgraph to worksgood. June 18: the daemon at the old checkout began logging “Failed to load graph” on thousands of consecutive ticks. The graph, and with it the January–June task ledger and its archive, was gone — clobbered during a period when the code’s own graph saves were non-atomic. The machinery survived the death of its host by being rebuilt: July 4, the coordination graph recreated from scratch, and the new [agency] configuration switched on with the automation wide open.

IV. Adulthood: the configuration of July 4

The July 4 configuration (quarantine series, `config.toml.2026-07-04T15-31-36Z`) is the machinery’s full portrait, every knob exposed:

```
auto_assign = true · auto_evaluate = true · flip_enabled = true · auto_rescue_on_eval_fail
= true · eval_gate_threshold = 0.7 · gate_confidence_threshold = 0.7 · gate_max_attempts
= 2 · gate_uncertain_policy = "escalate" · evolution_budget = 5 · evolution_interval
= 7200 · bizarre_ideation_interval = 10 · novelty_bonus_multiplier = 1.5 ·
ucb_exploration_constant = 1.4142135623730951
```

with four named agent-identities (assigner, creator, evaluator, evolver) as blake3 content hashes. Uncertainty flag: the pre-July-4 configuration of the old graph is not preserved anywhere we have found, so how much of this block predates the cutover is unrecorded.

This is the configuration the paper calls the *enforcement autonomy*: gates that admitted, escalated, rescued, and evaluated without a human decision. In its first week it lived through the provider churn (seven route changes, OpenRouter keyring adoption, the arrival of gpt-5.6-sol) — and by July 11, the automation had been walked back: `auto_assign = false`, `auto_evaluate = false`. By July 25 the block had churned again (`auto_evaluate = true` restored; the agent-identity hashes deleted from config). The knobs turned throughout the month; the constraint census records 43–46 births in July against 1–4 removals.

July also produced the machinery’s strangest crop: the verify-timeout experiment matrix — 76 tasks dispatched to test the verification system itself, mass-abandoned mid-month. The machinery testing itself until the testing broke.

V. The occupation of August

August began with the machinery’s own housekeeping misfiring: on August 1, the daemon “held overdue automatic archival” — mass-archiving tasks on startup, unbidden. The agents held it and made archival opt-in on August 2. On August 3 they built atomic graph saves with rescue-checkpointing — scar tissue from June’s corruption. On August 5, disk exhaustion triggered the atomic quarantine of the entire damaged graph (recovery tag: `recovery/overengineered-cutover-20260805`), and on August 7 the working repo was re-cloned from GitHub — the same day came the first retirement: “Retire legacy FLIP routing integration tests.”

The retirement commit is modest — two files, five lines, a baseline report — but it is the first time in the organization’s life that a governance artifact died by intent rather than by cutover. The census’s removal vocabulary had been extinct since April.

VI. Demobilization (September 13)

The September 13 configuration change — the operator’s hand, co-designed in the coordinator channel (“all these hardcore checks are hardcore and gummy”) — did not delete the machinery. The census is precise: **auto_evaluate** flipped **true** to **false**; **verify_autospawn_enabled** = **false**; **gates** were **demobilized**, **one decision away from re-armament**. Constraint existence and enforcement autonomy, the record now shows, were separate control surfaces all along.

What followed is the strangest entry in the biography. The constraint birth rate did not fall — September records 45 births, nearly matching August’s 46, most of them verify/prove-gated tasks around Pi route hardening and opaque execution. The machinery’s *language* redirected rather than retired: guarantee-language density, which ran 85.9 per 100 commits in April and collapsed to 8.1 in June, returned to 61.6 in September — but September’s words are *prove*, *exact*, *baseline*, *reconcile*. The same care, re-pointed from gating the work to proving the substrate. And on September 14, the organization wrote down its new constitution: “docs: define reliable-work operating principles.”

VII. Dismantling (September 14–18)

Then the retirement arc, at last: September 4, “make retired evaluation rows non-authoritative” (the cutover the record calls **harden-retired-evaluation-cutover**); September 16, “retire the WG model registry — Pi catalog is the single model source of truth” (governance externalized to the host harness); and through it all, FLIP itself was not killed but *domesticated* — September 18’s final entry in this window: “operator-framing docs + concierge framing print + FLIP glossary.” The machinery’s last act was to explain itself to humans.

Obituary

Here lies the governance machinery of wg: the Agency (February 6), FLIP (March 3), and the verify gates (April 11) — born to prevent failures, grown to 312 constraint topics on the fuel of those failures, never once effective at dying.

In life it carried a retirement strategy in its own source code, behind a test-only guard, guarded further by a rule that nothing under-tested could ever be retired — and evaluation was the first thing the burden broke.

It was not killed. It was demobilized by hand on September 13, its care redirected to proving the substrate, its surviving members explained in a glossary.

Cause of death, as recorded: the operator, with a configuration edit.

Survived by: atomic saves, a concierge, and a written constitution.

*Uncertainties flagged: (1) the pre-July-4 configuration of the original graph is unrecorded, so which [agency] knobs predate the cutover is unknown; (2) whether evolution cycles ever ran autonomously in production (evolution_budget 5 per 7200s cycle) is not directly evidenced in the artifacts examined — the config enabled the machinery, but we found no receipt of an autonomous evolution batch; (3) retirement existed in two non-operational layers: the deterministic threshold logic (**roles_below_threshold**) is **#[cfg(test)]**-only — its only complete implementation is test code — while the LLM-side capability existed solely as a prompt component (“retirement-identification”) in the evolver’s role description, with no receipt found of it ever being exercised; (4) the census’s 312-topic count is an upper bound (suffix clustering over-fragments; 164 high-confidence, 148 medium).*

Epilogue: what the book is for

The paper that grew alongside this book argues that knowledge hardened into rules, and rules hardened into vetoes. Recovery came from the reverse: keeping what was learned available for judgment while demobilizing

the rules—treating them as disposable implementations of knowledge, not as the knowledge itself. The earliest symptom was rising acceptance burden, not falling completion. The book exists because the paper needed the record to be true, and the record needed reading.

Three things the book adds that the paper cannot carry. First, texture: the constitution that forbids “a run that succeeds only after repeated manual retries” was written by the hand that had spent a month performing those retries. Second, the counterfactual: the same operator, the same tool, the same machines ran more than seventy-five other deployments that never cascaded—the disease needed the sustained organization, not the machinery. Third, the moral: every governance layer in this story—the gates, the archive policy, the gitignore entry from week three—was a rule written once and never reviewed, and the only mechanism that worked against them was a human reading the record—judging, demobilizing, retiring, externalizing, and only occasionally deleting.

The organization survives. Its state is durable, its machinery retired or externalized, its constitution written, its burden low. The book closes where the record stands.

Assembled by an agent fleet on lunaroute glm-5.3-flash and deepseek-4.1-flash, coordinated by pi, at the operator’s direction. Two of nine chapter workers produced narration instead of work and were rebriefed; the pattern is documented in the paper’s §4.2 and is the book’s final exhibit.